

Arduino IDE - LoRa V1.1

Date: 02/12/2025

Indice

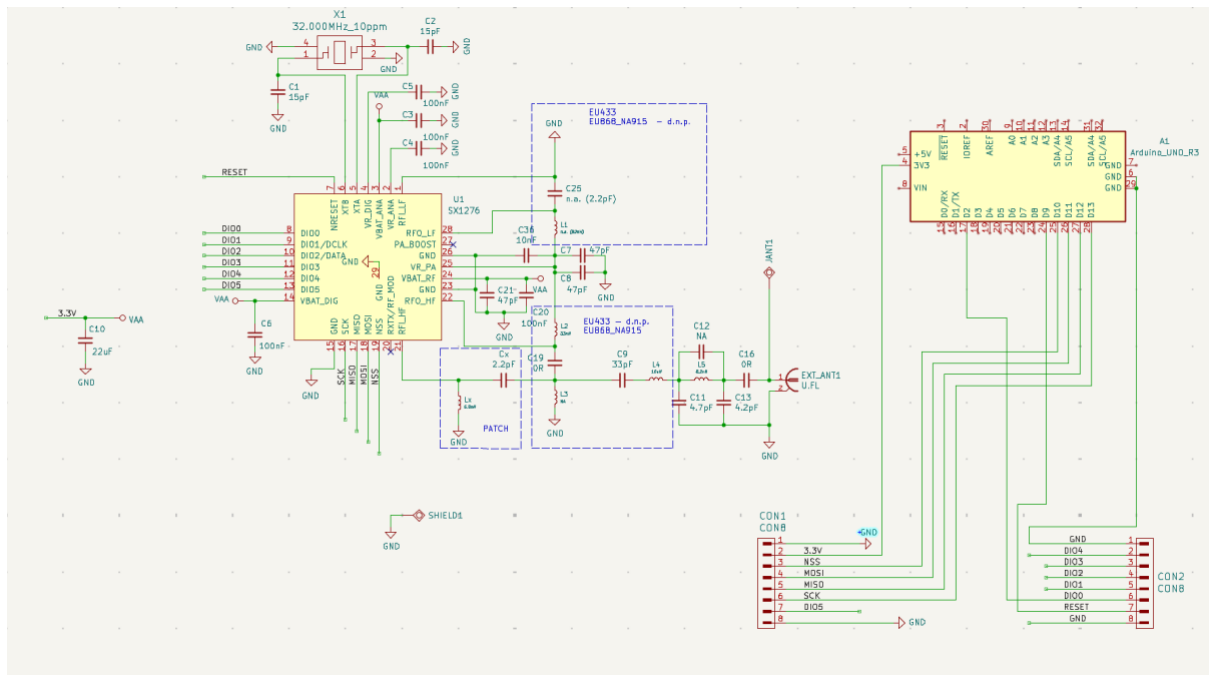
1.	Introdução ao Sistema	2
2.	Diagrama de Ligação de Hardware (Interface SPI)	2
3.	Implementação de Software (Código Arduino IDE para teste do módulo)	2
3.1.	Código Comum (Setup)	2
3.2.	Código do Transmissor (TX)	3
3.3.	Código do Recetor (RX)	4
3.4.	Código para teste individual do módulo LoRa com o arduino	5
4.	Plano de Desenvolvimento e Testes do Módulo de Comunicação LoRa	6
4.1.	Adaptação de Software e Integração de Hardware	6
4.1.1.	Transição da Plataforma Host (Arduino Uno → ESP32-S3)	6
4.2.	Otimização dos Parâmetros de Rádio (Spread Factor)	7
4.3.	Definição da Formatação da Mensagem	7
4.3.1.	Conteúdo do Payload e Estrutura	7
4.3.2.	Implementação de Segurança e Integridade	8
5.	Bibliografia	8
6.	Historico de versões:	8

1. Introdução ao Sistema

Este documento estabelece o protocolo de comunicação SPI para o Módulo OLIMEX LoRa868 (chip Semtech SX1276) utilizando o Arduino Uno como microcontrolador host. O módulo opera na frequência de 868 MHz (Brequência ISM Europeia).

2. Diagrama de Ligação de Hardware (Interface SPI)

A comunicação SPI é a interface nativa do chip SX1276. Devido à diferença de voltagem entre o Arduino Uno (5V) e o LoRa868 (3.3V), é fundamental o uso de um Conversor de Nível Lógico (Level Shifter).



3. Implementação de Software (Código Arduino IDE para teste do módulo)

A biblioteca LoRa.h é necessária para gerir o transmissor via SPI.

3.1. Código Comum (Setup)

```
#include <LoRa.h>
```

```

// (Ajustar conforme as ligações feitas no Arduino)

#define SS_PIN 10
#define RST_PIN 9
#define DI0_PIN 2

const long FREQUENCY = 868E6; // 868 MHz

void setup() {
  Serial.begin(9600);
  while (!Serial);
  Serial.println("Inicializando LoRa...");

  // Configurar Pinos e Iniciar Módulo na Frequência
  LoRa.setPins(SS_PIN, RST_PIN, DI0_PIN);
  if (!LoRa.begin(FREQUENCY)) {
    Serial.println("ERRO: Falha na inicialização do LoRa!");
    while (1);
  }

  // Definir a Potência de Transmissão (Máx. do LoRa868)
  LoRa.setTxPower(14);
  Serial.println("Módulo LoRa pronto.");
}

```

3.2. Código do Transmissor (TX)

```

int counter = 0;

void loop() {
  String message = "Pacote de Teste #" + String(counter);
  Serial.print("A enviar: ");
  Serial.println(message);

  // Sequência de Transmissão SPI:
  LoRa.beginPacket(); // Inicia o pacote
  LoRa.print(message); // Escreve os dados
}

```

```
LoRa.endPacket(); // Envia o pacote

counter++;
delay(5000);
}
```

3.3. Código do Recetor (RX)

```
void loop() {
  // A função parsePacket() verifica o pino DIO0 para sinal de receção.
  int packetSize = LoRa.parsePacket();

  if (packetSize) {
    Serial.print("Pacote Recebido: ");

    // Ler dados via SPI
    while (LoRa.available()) {
      Serial.print((char)LoRa.read());
    }

    // Leitura de Diagnóstico (Qualidade do Sinal)
    long rssi = LoRa.packetRssi();
    float snr = LoRa.packetSnr();

    Serial.println();
    Serial.print(" -> RSSI: ");
    Serial.print(rssi);
    Serial.println(" dBm");
    Serial.print(" -> SNR: ");
    Serial.print(snr);
    Serial.println(" dB");
  }
}
```

3.4. Código para teste individual do módulo LoRa com o arduino

```
#include <SPI.h>
#include <LoRa.h>

// Definição dos pinos (confirma se correspondem à tua montagem)
#define SS_PIN 10
#define RST_PIN 9
#define DIO0_PIN 2

void setup() {
  Serial.begin(9600);
  while (!Serial); // Espera o monitor abrir

  Serial.println("\n--- INICIO DO TESTE DE HARDWARE (OLIMEX LORA868) ---");

  // 1. Configurar os pinos
  LoRa.setPins(SS_PIN, RST_PIN, DIO0_PIN);

  // 2. Tentar iniciar o módulo
  // O LoRa.begin já verifica internamente se o chip está conectado e se a versão é 0x12
  if (!LoRa.begin(868E6)) {
    Serial.println("\n ✗ ERRO: O Arduino nao consegue falar com o modulo.");
    Serial.println("  Verifica:");
    Serial.println("    - Resistencias (Divisor de tensao) nos pinos 13, 11, 10, 9.");
    Serial.println("    - Alimentacao 3.3V.");
    Serial.println("    - Se os fios estao bem apertados na breadboard.");
    while (1);
  }

  // Se passou do "if" acima, o SPI está a funcionar!
  Serial.println(" ✓ SUCESSO! O modulo foi iniciado.");
  Serial.println("  Isto confirma que as soldaduras e o divisor de tensao estao a funcionar.");

  // --- TRUQUE EXTRA ---
  // Como a funcao da biblioteca é privada, vamos ler o registo manualmente via SPI
  // só para teres o prazer de ver o "0x12" no ecrã.

  digitalWrite(SS_PIN, LOW);    // Ativa o modulo
```

```

SPI.transfer(0x42 & 0x7F);    // Envia endereço 0x42 (modo leitura)
byte version = SPI.transfer(0x00); // Lê o valor de volta
digitalWrite(SS_PIN, HIGH);    // Desativa o modulo

Serial.print("3. Versao do Chip (Lida manualmente): 0x");
Serial.println(version, HEX);

if (version == 0x12) {
    Serial.println("👉 Confirmado: É um chip SX1276 saudavel.");
}
}

void loop() {
    // Nada a fazer
}

```

4. Plano de Desenvolvimento e Testes do Módulo de Comunicação LoRa

Este plano estabelece as próximas etapas críticas para a validação e integração do Módulo de Comunicação LoRa, em conformidade com as especificações do sistema ResQSense.

4.1. Adaptação de Software e Integração de Hardware

4.1.1. Transição da Plataforma Host (Arduino Uno → ESP32-S3)

A transição do ambiente de teste (Arduino Uno) para a plataforma de produção deve ser efetuada no ESP32-S3 WROOM-1 N8R8, que servirá como a Microcontroller Unit (MCU) Host.

- **Protocolo de Interface:** A comunicação com o Módulo LoRa868, que utiliza o chip Semtech SX1276, é realizada nativamente através da interface SPI.
- **Mapeamento de Pinos:** É mandatório redefinir as constantes SS_PIN, RST_PIN e DIO_PIN para os pinos GPIO correspondentes no ESP32-S3.
- **Compatibilidade de Tensão:** Reitera-se a atenção à diferença de voltagem (5V vs. 3.3V), embora o ESP32-S3 opere a 3.3V, a compatibilidade de E/S deve ser verificada para o módulo LoRa868, para mitigar o risco de hardware burnout.

4.2. Otimização dos Parâmetros de Rádio (Spread Factor)

O Módulo de Comunicação é vital para o sistema, devendo garantir a robustez em ambientes complexos. A otimização do Spread Factor (SF) é crucial para equilibrar a taxa de dados e o alcance.

Parâmetro	Teste de Limitação e Objetivo	KPI de Sucesso (KPI)
Spread Factor (SF)	Testar a variação do SF (e.g., SF7 a SF12) para determinar o valor que minimiza o Time On Air (ToA) , mantendo a cobertura em ambientes com obstáculos (NLOS).	O ToA deve ser o menor possível para assegurar o requisito de Latência de Alerta < 3 segundos .
Taxa de Entrega de Pacotes (PDR)	Medir a percentagem de mensagens recebidas sem perda sob diferentes distâncias e condições de obstáculo.	Perda de pacotes deve ser < 5%.
Taxa de Transmissão	O SF e a Largura de Banda (BW) devem ser compatíveis com a taxa de transmissão contínua de dados a 0.1 Hz (a cada 10 segundos).	Transmissão contínua a 0.1 Hz sem sobrecarga da rede.

4.3. Definição da Formatação da Mensagem

A formatação do quadro de mensagem (message frame) é To Be Determined (TBD) e deve garantir a segurança e integridade dos dados, além de minimizar o payload para otimizar o tempo de transmissão.

4.3.1. Conteúdo do Payload e Estrutura

O pacote deve agregar os seguintes dados provenientes dos módulos:

- **Identificação:** Identifier (ID) do Operacional.
- **Localização:** Posição absoluta (GNSS) e relativa (UWB/IMU).
- **Biométricos:** Frequência Cardíaca (BPM), Níveis de SpO2 e Temperatura Corporal.
- **Estado:** Detecção do estado físico (e.g., “fallen”, “ok”) e Alertas de Alta Prioridade (e.g., 'Man Down', SpO2 Crítico).

4.3.2. Implementação de Segurança e Integridade

A estrutura da mensagem final deve incorporar os seguintes mecanismos de segurança, antes de ser enviada ao transmissor LoRa:

1. **Encriptação:** O payload deve ser encriptado usando o algoritmo AES-128-CTR.
2. **Integridade:** O pacote deve incluir um CRC (Cyclic Redundancy Check) para permitir ao recetor a deteção de erros durante a transmissão.

Nota: A definição de um formato binário ou de um formato de texto compacto (e.g., JSON otimizado com chaves curtas) deve ser priorizada para maximizar a eficiência do payload.

5. Bibliografia

Documentação LoRa:

(<https://randomnerdtutorials.com/esp32-lora-rfm95-transceiver-arduino-ide/>)

<https://github.com/OLIMEX/LoRa-868-915/tree/main/SOFTWARE/Arduino>

<https://www.makerhero.com/blog/comunicacao-lora-com-arduino-p2p/>

Documentação importante do ESP:

<https://www.botnroll.com/pt/esp32/5227-esp32-s3-placa-da-waveshare-com-esp32-s3-wroom-1-n8r8-240mhz-dual-core-usb-c-headers-soldados.html>

<https://www.waveshare.com/wiki/ESP32-S3-DEV-KIT-N8R8>

6. Histórico de versões

Versão 1.0- versão inicial da documentação LoRa

Versão 1.1- adição do esquema da interface LoRa -> Arduino e adição do código teste de cada módulo individual